

LightNobel and FPGA Triangle-Multiplication Kernels: Technical Summary

This document summarizes (1) the operation of LightNobel (Han, Choi, Kim, ISCA 2025, arXiv 2505.05893), a hardware-software co-designed accelerator for protein structure prediction models (PPMs); (2) our proposed optimization steps (v0-v3) for the Triangle Multiplication einsum on a spatial accelerator; and (3) the analytical methods used to evaluate them. Statements marked (*our interpretation*) are not stated explicitly in the paper.

1. Background: ESMFold and the Pair Representation

1.1 Pipeline

ESMFold, the baseline model used by LightNobel, predicts a 3D structure from an amino-acid sequence of length L in three stages:

Stage	Function	Main data
Input embedding	ESM-2 (3B) protein language model	sequence representation $s : L \times 1024$
Folding trunk	48 Protein Folding Blocks, up to 4 recycling iterations	s and pair representation $z : L \times L \times 128$
Structure module	predicts atom coordinates; distogram head from z	3D structure

Each element $z[i][j]$ (a *token* of the pair representation) is a 128-channel vector describing the relation between residues i and j . Every block updates the state residually ($z \leftarrow z + \Delta z$) in the following order: sequence attention, sequence transition, sequence-to-pair projection, Triangle Multiplication (outgoing), Triangle Multiplication (incoming), Triangle Attention (starting node), Triangle Attention (ending node), pair transition.

1.2 Triangle Multiplication

For the outgoing variant, with channel dimension $C = 128$, the update is:

```
z_ln = LayerNorm_in(z)
a     = mask ⊙ sigmoid(W_ag · z_ln) ⊙ (W_ap · z_ln)           (per token, 128 → 128)
b     = mask ⊙ sigmoid(W_bg · z_ln) ⊙ (W_bp · z_ln)
x[i][j][c] = Σ_k a[i][k][c] · b[j][k][c]                     (einsum "ikc,jkc->ijc")
g     = sigmoid(W_g · z_ln)
Δz    = g ⊙ (W_z · LayerNorm_out(x))
```

All operations except the einsum are token-local: each token is processed using only its own 128 values and the weights. The einsum is the only cross-token operation. For each channel c it is an $L \times L$ matrix product, $x_c = a_c \cdot b_c^T$; the 128 channels are independent (batched matrix multiplication). The incoming variant computes $x[i][j][c] = \sum_k a[k][i][c] \cdot b[k][j][c]$, i.e. $a_c^T \cdot b_c$.

Per output token, the einsum costs $2 \cdot L \cdot C$ operations (256,000 at $L = 1,000$), and the six 128×128 linear layers cost $6 \cdot 2 \cdot C^2 = 196,608$ operations. The einsum therefore dominates TriMul arithmetic as L grows.

Operation types in one Triangle Multiplication, per output token:

Operation	Type	Operations per token	Data dependence
LayerNorm_in, LayerNorm_out	normalization (reduction over 128 channels)	$O(C)$	token-local
6 linear layers (128×128)	matrix-vector	$6 \cdot 2C^2 = 196,608$	token-local, shared weights
sigmoid, gating, mask	element-wise	$O(C)$	token-local
einsum	batched matrix multiplication	$2 \cdot L \cdot C$	reads row i of a and row j of b

1.3 Triangle Attention and scaling

Triangle Attention applies attention along rows (or columns) of z . Its attention logits have shape $L \times L \times L$ per head, so they grow cubically; all other pair-representation tensors grow quadratically. For the starting-node variant, with $H = 4$ heads of dimension 32:

```

x = LayerNorm(z)
q, k, v = W_q x, W_k x, W_v x                (per token, split into H heads)
β[h][j][k] = (W_β x)[j][k][h]                (pair bias)
s[i][h][j][k] = q[i][j][h] · k[i][k][h] / √32 + β[h][j][k]
o[i][j][h] = Σ_k softmax_k(s[i][h][j][k]) · v[i][k][h]

```

The ending-node variant applies the same computation to the transpose of z .

Tensor	Shape	Growth
Sequence representation	$L \times 1024$	$O(L)$
Pair representation and TriMul intermediates	$L \times L \times 128$	$O(L^2)$
Triangle Attention logits (per head)	$L \times L \times L$	$O(L^3)$

The LightNobel paper reports that at $L = 2,034$ the activations are already $24.15 \times$ larger than the weights and require 144 GB, exceeding a single GPU. Its execution-time profile on

an H100 without chunking (Fig. 3) shows the shift in bottleneck:

Share of total time	Short protein (R0271)	Long protein (T1269)
Protein Folding Block	83.8%	94.5%
Pair-representation dataflow	69.4%	91.9%
Triangle Multiplication	36.1%	14.5%
Triangle Attention	29.0%	75.9%

2. LightNobel

2.1 Overview

LightNobel targets the Protein Folding Block only; ESM-2 and the structure module are not mapped to the accelerator. The design rests on four observations:

1. For long sequences, execution time is concentrated in the pair-representation dataflow, especially Triangle Attention.
2. Memory is dominated by activations, not weights.
3. Values vary little across the channels of one token but strongly across tokens, and outliers concentrate in specific tokens (correlated with distogram patterns). Token-wise quantization therefore fits better than channel-wise quantization.
4. Activations at different positions in the block have different statistics (e.g. mean absolute value about 82 before LayerNorm on the residual path vs. about 4 elsewhere).

The contributions are: Token-wise Adaptive Activation Quantization (AAQ) in software; a Reconfigurable Matrix Processing Unit (RMPU), a Versatile Vector Processing Unit (VVPU) and a Token Aligner in hardware; and a token-wise dataflow that avoids storing large intermediates, including the L^3 attention logits.

2.2 Token-wise Adaptive Activation Quantization (AAQ)

Each token t is quantized symmetrically with its own scale:

$$\begin{aligned} s_t &= \max_c |x_t[c]| / (2^{(b-1)} - 1) & (b = 4 \text{ or } 8) \\ q_t[c] &= \text{round}(x_t[c] / s_t) \\ \hat{x}_t[c] &= s_t \cdot q_t[c] \end{aligned}$$

Outliers are the k values of largest magnitude in each token. They are selected at run time with a top- k operation and stored in INT16 at original precision; the remaining values (inliers) are quantized. Activations are divided into three groups by their position in the block:

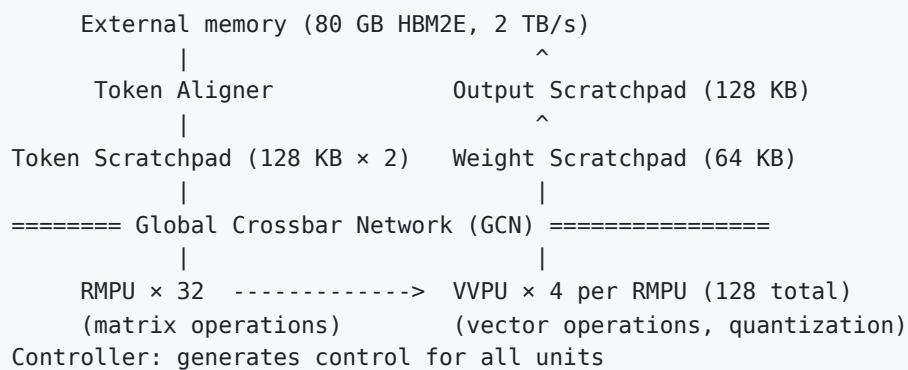
Group	Activations	Mean abs. value	Avg. outliers per token	Format
A	before LayerNorm, on the residual path	82.14	2.31	INT8 inliers + 4 INT16 outliers
B	after LayerNorm, before linear layers	4.05	1.69	INT4 inliers + 4 INT16 outliers
C	all other activations (e.g. linear outputs)	3.85	0.64	INT4, no outlier handling

Additional design points:

- Weights are not quantized; they are stored in INT16. The baseline (ESMFold) uses FP16 for weights and activations.
- Symmetric quantization is used because, with outlier handling, its RMSE is only 9.76% higher than asymmetric quantization (27.35% without outlier handling), and it is cheaper in hardware.
- The per-group settings were chosen by design-space exploration over precision and outlier count, evaluated by TM-score and data size. Rejected alternatives include: INT4 for group A (needs 32 or more outliers to keep accuracy), fewer than 4 outliers for group A (TM-score drops), INT8 for group B (accurate but larger), and channel- or tensor-wise granularity.
- Storage format per token: inliers, then outliers, then the scale, then outlier indices. Tokens are packed into blocks sized to the memory-channel bandwidth; compressed token length differs by group.

In the Triangle Multiplication, `z` belongs to group A, `z_ln` to group B, and `a`, `b` to group C (INT4).

2.3 Architecture and execution model



The units are time-shared rather than wired as a fixed pipeline for one layer:

- The RMPU executes all matrix-type operations: linear layers, the TriMul einsum, $Q \cdot K^T$ and attention $\cdot V$.
- The VVPU executes element-wise and vector operations: LayerNorm, sigmoid, gating, residual addition, softmax, and run-time quantization.
- The GCN routes data between units in any order (e.g. VVPU $\rightarrow$ RMPU $\rightarrow$ VVPU).

A model layer executes as a sequence of passes. In each pass, the Token Aligner reads a block of packed tokens from HBM, decodes it and writes one token per scratchpad row. The Token Scratchpad is double-buffered, so the next block loads while the current one is processed. Weights for the pass are held stationary in the Weight Scratchpad. The VVPU performs any preprocessing (e.g. LayerNorm), the RMPU performs matrix operations on up

to 20 tokens at a time, and results stream directly into the VVPU for post-processing and run-time quantization. Quantized tokens are written back through the Output Scratchpad. Latency is determined by the slowest pipeline stage (RMPU, VVPU, or memory).

Weight-stationary dataflow is chosen because a 128×128 INT16 weight matrix is only 32 KB but is reused by up to L^2 tokens.

2.4 RMPU: bit-decomposed multi-precision matrix unit

Mixed INT4/INT8/INT16 operands would leave dedicated per-precision multipliers idle. The RMPU instead uses one kind of small multiplier (following Bit Fusion) and composes larger products from 4-bit chunks:

$$A = \sum_i a_i \cdot 2^{(4i)}, \quad B = \sum_j b_j \cdot 2^{(4j)}$$

$$A \cdot B = \sum_i \sum_j (a_i \cdot b_j) \cdot 2^{(4(i+j))}$$

The shifts are implemented without multipliers. An $\text{INT4} \times \text{INT16}$ product needs 4 chunk products, $\text{INT8} \times \text{INT16}$ needs 8, and $\text{INT16} \times \text{INT16}$ needs 16.

In two's complement, only the most significant chunk is signed; the lower chunks are unsigned. The Reconfigurable Data Aligner (RDA) sign-extends the top chunk and zero-extends the others to 5 bits. Every chunk then fits one 5-bit signed format (-16 to 15), so the multipliers take 5-bit signed inputs and a single multiplier type handles all chunks.

Hierarchy:

Level	Composition	Multipliers
PE	16 multipliers + shifters + 16-to-1 adder tree	16
PE Lane	8 PEs	128
PE Cluster	20 PE Lanes + Dynamic Accumulation Logic (DAL)	2,560
RMPU Engine	4 PE Clusters (80 lanes, 640 PEs)	10,240

A 128-element dot product of a group-C token (128 INT4) with INT16 weights needs $128 \times 4 = 512$ multipliers (4 lanes); a group-B token ($124 \text{ INT4} + 4 \text{ INT16}$) needs $124 \times 4 + 4 \times 16 = 560$ (5 lanes). A cluster of 20 lanes, the least common multiple of 4 and 5, is fully utilized in both cases (5 or 4 tokens at a time).

The DAL handles scaling. Inlier partial sums must be multiplied by the token scale; outlier partial sums (INT16 originals) must not be. For 4-lane tokens, the lane sums are added and scaled once. For 5-lane tokens, the four inlier lanes are added and scaled, and the outlier lane is added afterwards. An arbiter and reconfigurable adder trees switch between 4-to-1 and 5-to-1 reduction. Results can be tapped at several levels of the adder tree: 2-PE sums

for head dimension 32 in attention, 4- or 5-lane sums for quantized linear layers, and 8- or 16-lane sums for unquantized operands.

2.5 VVPU: vector unit and run-time quantization

Each VVPU contains 128 SIMD lanes (one per channel of a token, each with a 16-bit ALU, a local scratchpad and a two-level exponent lookup table), a Scalar Support Unit (SSU) for reductions and format handling, and a Local Crossbar Network (LCN) for data exchange between lanes.

Top-k selection uses a bitonic sorting network. It is a fixed network of compare-and-swap elements with data-independent control; for 128 values it needs $7 \cdot 8 / 2 = 28$ stages of 64 comparators, and can terminate early for top-k. Original indices are tracked to record outlier positions, and $k = 1$ yields the maximum used for softmax and scale computation.

Run-time quantization proceeds in four steps: (1) top-k selection of outliers and the scale, (2) scaling and rounding of inliers, (3) reordering into the storage format through the LCN, and (4) final alignment by the SSU. Full-precision values are never written to external memory.

The paper's design-space exploration (Fig. 12) shows that latency saturates at 4 VVPUs per RMPU and at 32 RMPUs. Beyond these points the added units are not on the critical path, or the memory system cannot supply more data.

2.6 Token-wise multi-head attention

To avoid storing the L^3 logits, LightNobel computes attention token-wise in a manner similar to FlashAttention. For each query, scores are processed in blocks with running statistics:

```
m_new = max(m, max(s))
ℓ      = ℓ · exp(m - m_new) + Σ exp(s - m_new)
o      = o · exp(m - m_new) + Σ exp(s - m_new) · V
m      = m_new
output = o / ℓ
```

On the hardware, the RMPU computes $Q \cdot K^T$ per head in parallel, the VVPU dequantizes and accumulates, and softmax for one block overlaps with $Q \cdot K^T$ for the next; V is then applied and the result written back. The small hidden dimension (128) allows many tokens to reside on chip simultaneously, which makes the token-wise approach efficient for PPMs.

2.7 Mapping of Triangle Multiplication (*our interpretation*)

The paper does not list per-operation schedules. Based on operation dependencies and the described capabilities, TriMul executes in three passes:

Pass	Scope	Operations
1	token-local, over (i, k)	read z → VVPU LayerNorm → RMPU four linear layers → VVPU sigmoid, gating, mask, INT4 quantization → write a , b
2	cross-token	read a , b → RMPU einsum → VVPU dequantization and accumulation
3	token-local, over (i, j)	VVPU LayerNorm_out → RMPU w_z , w_g → VVPU sigmoid, gating, residual, INT8 + outlier quantization → write z

a and b must be written to memory between passes 1 and 2 because the einsum reads entire rows across tokens.

2.8 Evaluation methodology and results

The paper evaluates the design with the following tools:

- A Python cycle-accurate simulator, cross-validated against RTL simulation (mean error 3.30%, all within 5%).
- SystemVerilog RTL synthesized with Synopsys Design Compiler at 28 nm and 1 GHz.
- CACTI 7.0 and a memory compiler for SRAM, scaled to 28 nm.
- Ramulator for 80 GB HBM2E at 2 TB/s.

The workload is ESMFold (ESM-2 3B) on CAMEO, CASP14, CASP15 and CASP16, compared with NVIDIA A100 and H100.

Metric	Result
Accuracy	TM-score change < 0.001
Speedup	up to 8.44× (A100), 8.41× (H100); 3.85–8.44× and 3.67–8.41× vs. GPUs with chunking
Power efficiency	up to 37.29× (A100), 43.35× (H100)
Peak memory	up to 120.05× lower than unchunked GPU; 1.26–5.05× lower than chunked GPU
Maximum length	9,945 residues within 80 GB (1.45× the longest CASP16 protein, 6,879)
Area / power	178.80 mm ² , 67.8 W

The area and power breakdown (Table 2) is dominated by data movement. The crossbars (128 LCNs + GCN, about 125.7 mm²) take 70.28% of area and 67.95% of power, while the RMPU engines take 18.20% of area. Within one VVPU, the LCN (0.785 mm²) is about 7× larger than the 128 SIMD lanes (0.115 mm²).

Memory savings come from two separate mechanisms:

- **Quantization.** With the dataflow unchanged, total memory drops from 121.39 GB to 73.50 GB (1.65×; T1169, 3,364 residues, Table 1).

- **Not materializing intermediates.** In particular, removing the L^3 attention logits accounts for most of the 120× peak reduction relative to an unchunked GPU.

The pair representation still grows as L^2 .

2.9 Related approaches

Approach	Examples	Effect	Limitation
GPU system optimization	FastFold, ScaleFold	faster training and inference	memory still grows as L^2
Chunking	AlphaFold/ESMFold chunk option, AutoChunk	lower peak memory	slower; repeated reads
Model quantization	MEFold (weights), PTQ4Protein (per-tensor INT8)	smaller data	weight-only quantization has limited effect on activations; accuracy loss at low precision
LLM quantization	SmoothQuant, LLM.int8(), AWQ	outlier separation	single inlier precision; activation quantization is often slower on GPUs
Fused GPU kernels	OpenFold3 fused TriMul	intermediates not materialized	hand-written kernels, GPU-specific
Dedicated accelerator	LightNobel	quantization, specialized units and token-wise dataflow	simulated, not fabricated

3. Proposed Kernel Optimizations for the Triangle-Multiplication Einsum (v0-v3)

This section describes the optimization steps in platform-independent terms. Each step is expressed through the following parameters:

Symbol	Meaning
L	sequence length
C	channels per token (128)
e	bytes per element (4 for FP32)
T_I, T_J	output tile size along i and j
W	elements delivered per memory transfer cycle (interface width / e)
P	multiply-accumulate (MAC) operations per cycle
α	memory access latency per request (cycles)
δ	pipeline depth of the MAC datapath (cycles)

3.1 Problem formulation and data layout

The kernel computes $z[i][j][c] = \sum_k a[i][k][c] \cdot b[j][k][c]$ for all $i, j < L$ and $c < C$, i.e. C independent $L \times L$ matrix products $z_c = a_c \cdot b_c^T$. The total work is $L^2 \cdot C$ outputs, each requiring L MACs, i.e. $2 \cdot L^3 \cdot C$ operations.

All three tensors use the layout `[row][col][channel]`, the native layout of the model. The C channels of one token are contiguous ($C \cdot e = 512$ bytes for FP32). Row i of `a` and row j of `b` are therefore sequences of L contiguous tokens. Both operands of the einsum are read along rows, so neither needs to be transposed.

3.2 v0: baseline

Each output element $z[i][j][c]$ is computed independently with the reduction over k innermost. Every MAC fetches one element of `a` and one of `b` from external memory, and nothing is kept on chip except a single accumulator.

- **Traffic.** $2 \cdot L^3 \cdot C$ element reads, with no reuse: each element of `a` is re-read for every j and each element of `b` for every i .
- **Access pattern.** Consecutive k iterations access addresses C elements apart. Requests cannot be merged into bursts, so each element incurs the full latency α .
- **Arithmetic intensity.** 2 operations per $2e$ bytes = 0.25 FLOP/byte for FP32.

3.3 v1: loop tiling with on-chip buffers

Idea. The output space is partitioned into $T_I \times T_J$ tiles, and the accumulators of one tile ($T_I \cdot T_J \cdot C$ values) are kept on chip for the whole reduction (output-stationary). The reduction over k is moved inside the tile. For each k :

1. T_I tokens $a[i_0 \dots i_0+T_I-1][k]$ and T_J tokens $b[j_0 \dots j_0+T_J-1][k]$ are loaded into on-chip buffers.
2. All $T_I \cdot T_J \cdot C$ products are computed from these buffers.

Each loaded element of a is used by T_J outputs and each element of b by T_I outputs.

Reuse. Off-chip reads drop from $2 \cdot L^3 \cdot C$ to $L^3 \cdot C \cdot (1/T_I + 1/T_J)$, a factor of 8 for 8×8 tiles.

Contiguous access. Each load transfers one complete token, i.e. C contiguous elements. A token is fetched as one burst, so the latency α is paid once per token instead of once per element.

Per-k cost with one element transferred and one MAC per cycle:

$$\begin{aligned} t_k(v1) &\approx (T_I + T_J) \cdot C + T_I \cdot T_J \cdot C + \text{latency terms} \\ &= 2,048 + 8,192 = 10,240 \text{ cycles} \quad (T_I = T_J = 8, C = 128) \end{aligned}$$

Computation dominates (80%), so the next step must increase the MAC rate, not only the transfer rate.

3.4 v2: wide interface and vectorization

Idea. The C channels of the einsum are independent (no reduction over c) and contiguous in memory. The kernel can therefore process W channels per transfer and P channels per MAC cycle without any data reorganization. v2 widens three parts of the datapath consistently:

1. **Transfer width.** W elements per memory transfer (e.g. 512-bit transfers carry $W = 16$ FP32 values). One token then takes C / W transfers.
 2. **On-chip buffers.** Buffers are partitioned into W ($= P$) independent banks along the channel dimension, so P operands can be read per cycle.
 3. **Datapath.** P parallel MAC units process P channels of the same (i, j, k) triple per cycle.
- a and b are read through separate input streams, so their tile loads proceed concurrently.

Per-k cost:

$$\begin{aligned} t_{\text{load}} &= \max(T_I, T_J) \cdot C / W + \alpha \\ t_{\text{mac}} &= T_I \cdot T_J \cdot C / P + \delta \\ t_k(v2) &= t_{\text{load}} + t_{\text{mac}} \\ \text{Example } (T = 8, C = 128, W = P = 16): & \quad t_{\text{load}} = 64 + \alpha, \quad t_{\text{mac}} = 512 + \delta \end{aligned}$$

Relative to v1, the transfer term shrinks by W and the compute term by P . With $\alpha \approx 80$ and $\delta \approx 15$ cycles (illustrative values), t_k drops from about 10,240 to about 671 cycles, roughly 15 $\times$. The ratio is below 16 because α and δ do not scale with width.

3.5 v3: double buffering

Idea. In v2 the MAC units are idle while a tile slice is loaded (t_{load} of every t_k cycles). v3 overlaps the loading of $k+1$ with the computation of k :

- **Two buffer sets.** Two sets of input buffers (set 0 and set 1) are allocated.
- **Schedule.** Before the reduction, slice $k = 0$ is loaded into set 0. At each even k , slice $k+1$ is loaded into set 1 while set 0 is consumed; at each odd k the roles swap.
- **Independence.** Loading and computation access disjoint buffers, so they can run concurrently as independent hardware units.
- **Why two sets.** A single buffer would create a write-after-read hazard: the next load would overwrite data still being read by the MAC units.
- **Cost.** Input-buffer memory doubles. Traffic, transfer width and MAC count are unchanged.

```
v2:  [load k=0][ mac k=0 ][load k=1][ mac k=1 ][load k=2][ mac k=2 ] ...
v3:  [load k=0][ mac k=0 ][ mac k=1 ][ mac k=2 ][ mac k=3 ] ...
      [load k=1] [load k=2] [load k=3]
```

Cost per tile:

```
v2:  L · (t_load + t_mac)
v3:  t_load + L · max(t_load, t_mac)
speedup (large L) = (t_load + t_mac) / max(t_load, t_mac) ≤ 2
Example (t_load ≈ 144, t_mac ≈ 527): 671 / 527 ≈ 1.27
```

The benefit is bounded by the fraction of time spent loading (about 21% in the example), following Amdahl's law. It reaches the maximum of 2 only when $t_{\text{load}} \approx t_{\text{mac}}$.

3.6 Summary of v0-v3

Version	Idea	Off-chip reads (elements)	MACs per cycle	Time per reduction step
v0	baseline, reduction innermost	$2 L^3 C$	1	latency-bound per element
v1	tiling, on-chip accumulators and buffers, token-contiguous loads	$L^3 C (1/T_I + 1/T_J)$	1	$(T_I + T_J) C + T_I T_J C$
v2	wide transfers (W), vectorized MACs (P) over channels	same as v1	P	$t_{\text{load}} + t_{\text{mac}}$

Version	Idea	Off-chip reads (elements)	MACs per cycle	Time per reduction step
v3	double buffering	same as v1	P	$\max(t_{\text{load}}, t_{\text{mac}})$

4. Computation Methods

4.1 Execution-time model

```
no overlap:  T ≈ T_compute + T_memory
overlap:     T ≈ max(T_compute, T_memory)
T_compute = operations / (operations per cycle × f_clk)
T_memory   = bytes / bandwidth + α × number of non-overlapped requests
```

Each optimization acts on one term:

Technique	Term affected	Version
Tiling	bytes ↓	v1
Contiguous (burst) access	non-overlapped requests ↓	v1
Wide transfers	bandwidth ↑	v2
Parallel MAC units	operations per cycle ↑	v2
Double buffering	sum → max	v3

4.2 Off-chip traffic

```
v0:  R_0 = 2 · L3 · C
v1:  R_1 = (L/T_I)(L/T_J) · L · (T_I + T_J) · C = L3 · C · (1/T_I + 1/T_J)
R_0 / R_1 = 2 / (1/T_I + 1/T_J) = 8 (8×8), 16 (16×16), 32 (32×32)
```

Output writes ($L^2 \cdot C$) are identical in all versions and negligible compared with the reads for large L .

4.3 Arithmetic intensity and roofline

```
AI = FLOPs / bytes = 2 L3 C / (e · R)
v0:  AI = 1 / e = 0.25 FLOP/byte (FP32)
v1-v3: AI = 2 T_I T_J / (e (T_I + T_J)) = 2 (8×8), 4 (16×16), 8 (32×32) for e = 4
attainable performance = min(peak compute, AI × bandwidth)
```

Compute-bound condition. For the vectorized design, peak compute is $2 \cdot P \cdot f_{\text{clk}}$ and the two input streams supply $2 \cdot W \cdot e \cdot f_{\text{clk}}$ bytes/s. The kernel is compute-bound when

$$2 P f / AI \leq 2 W e f \quad \Leftrightarrow \quad P / AI \leq W \cdot e$$

With $P = W = 16$, $e = 4$ and $AI = 2$, the left side is 8 and the right side 64, so the tiled kernel is compute-bound and larger tiles are not needed for bandwidth. v0 ($AI = 0.25$, no bursts) is latency- and memory-bound.

4.4 Cycle model and calibration method

```

cycles_total = (L/T_I)(L/T_J) · cycles_tile
cycles_tile  = t_init + L · t_k + t_store
t_load       = max(T_I, T_J) · C/W + α
t_mac        = T_I · T_J · C/P + δ
t_k          = t_load + t_mac                (v2)
t_k          = max(t_load, t_mac)           (v3, plus one initial t_load per tile)
t_init       = t_store ≈ T_I · T_J · C/P + σ

```

The structural terms follow from the loop bounds. The constants α , δ and σ depend on the implementation platform and are calibrated from one reference configuration (synthesis estimate or measurement). The model is then validated on configurations not used for calibration (other tile sizes, v3), with a target error below 10%. The model does not capture contention when several streams share one memory channel.

4.5 On-chip memory

```

accumulators = T_I · T_J · C · e          (8×8: 32 KB; 16×16: 128 KB; 32×32: 512 KB)
input buffers = (T_I + T_J) · C · e       (8×8: 8 KB; doubled by double buffering)

```

Arithmetic intensity grows linearly with tile size, while accumulator storage grows quadratically. The tile size is chosen at the knee of the cycles-versus-on-chip-memory Pareto front.

4.6 Capacity model

Memory requirement scales with L^2 , so the maximum sequence length scales with the square root of the available capacity M :

```

memory = L2 · (bytes per pair position)
L_max  = sqrt(M / bytes per pair position)

```

The relative gain is independent of M . With an FP16 baseline (the precision used by ESMFold and by the LightNobel comparison):

Configuration	Bytes per L^2 position	Relative $L_{\max}$
Unfused TriMul (≈ 7 live $L \times L \times 128$ tensors, FP16)	$7 \times 256 = 1,792$	1.00×
Fused producer/consumer, FP16 z , INT8 a , b (per-token scale)	$2 \times 256 + 2 \times 132 = 776$	$\text{sqrt}(1,792 / 776) \approx 1.52\times$
Same with INT4 a , b	$2 \times 256 + 2 \times 68 = 648$	$\text{sqrt}(1,792 / 648) \approx 1.66\times$

An INT8 token occupies 128 B plus a 4-byte scale, and an INT4 token 64 B plus a 4-byte scale. The fused configurations assume that a and b are produced token-by-token (LayerNorm, linear layers, gating and quantization fused), that g is recomputed in the consumer instead of stored, and that only z , the output, a and b are materialized. Because L grows with the square root of the memory reduction, a $2.3\times$ reduction in bytes per position yields about $1.5\times$ in L .